অধ্যায় ২৩

JavaScript Performance Optimization, Memory Management, Profiling এবং Web Application Performance

Hands-on Project: TravelSearch Explorer — ১২,০০০ রেকর্ডের Search ও Virtual Scrolling

---

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) অনুমানের বদলে Performance Measure করতে, (২) Time Complexity ও বাস্তব Benchmark-এর পার্থক্য বুঝতে, (৩) Chrome/Edge DevTools-এর Performance ও Memory Panels ব্যবহার করতে, (৪) Rendering ও Long Task-এর সমস্যা শনাক্ত করতে, (৫) Debounce, Throttle, Caching এবং Virtualization প্রয়োগ করতে, (৬) Memory Leak এড়াতে এবং (৭) Performance Regression-এর Test Plan লিখতে পারবে।

২৩.১ Performance Optimization মানে কী?

দ্রুত কোড লেখা আর দ্রুত চলা কোড এক নয়। কোনো Algorithm-এর CPU Time কম হলেও যদি ১০,০০০টি DOM Element একসঙ্গে তৈরি করে, Browser Layout ও Paint ধীর হতে পারে। আবার অত্যধিক Cache Memory বাড়িয়ে দিতে পারে। তাই Performance বিচার করতে হয় **User Experience, CPU, Memory, Network এবং Rendering** —সবকিছু মিলিয়ে।

Optimization-এর প্রথম নিয়ম: **Measure → Identify Bottleneck → Change One Thing → Measure Again**। পরিমাপ ছাড়া "এটা Faster" বলা অনুমান।

গুরুত্বপূর্ণ Metric

Metric	কী বোঝায়	কোথায় মাপবে
Interaction to Next Paint (INP)	User Interaction-এর প্রতিক্রিয়া	Field data / Web Vitals
Largest Contentful Paint (LCP)	বড় Content দৃশ্যমান হতে সময়	Lighthouse / Field data
Cumulative Layout Shift (CLS)	Layout-এর অপ্রত্যাশিত নড়াচড়া	Lighthouse / Field data
Long Task	Main Thread ৫০ ms-এর বেশি ব্যস্ত	DevTools Performance
JS Heap Size	JavaScript Object Memory	DevTools Memory
Filter/Sort Duration	Business Operation কত সময় নেয়	`performance.now()`

Lighthouse-এর Lab Result ও Real User Field Data এক নয়। একটি Fast Desktop-এর Benchmark সব Mobile Device-এর অভিজ্ঞতা বোঝায় না।

২৩.২ Complexity বনাম বাস্তব সময়

```javascript
const items = Array.from({ length: 10000 }, (_, i) => i);
const found = items.find(x => x === 9999); // worst case O(n)
const doubled = items.map(x => x * 2); // O(n)
const pairs = [];
for (let i = 0; i < 100; i++) {
  for (let j = 0; j < 100; j++) pairs.push([i, j]); // O(n²)
}
```

Big-O Input বাড়লে Growth Rate কেমন হয়, তা বোঝায়; Millisecond নির্ধারণ করে না। একই O(n) Operation ভিন্ন Hardware, Engine ও Data Shape-এ ভিন্ন সময় নিতে পারে।

`Map` ও `Set` কখন কাজে লাগে?

```javascript
const byId = new Map(products.map(p => [p.id, p]));
const product = byId.get(42); // average-case efficient lookup
const uniqueTags = [...new Set(tags)];
```

একবার Index তৈরির খরচ আছে। মাত্র পাঁচটি Item-এর জন্য `Map` বানানো অপ্রয়োজনীয় হতে পারে; হাজারবার Lookup হলে লাভ হতে পারে।

২৩.৩ Performance Measurement

```javascript
const start = performance.now();
const result = expensiveOperation();
const duration = performance.now() - start;
console.log(`Operation took ${duration.toFixed(2)} ms`);
```

একবার মাপলে JIT Warm-up, Garbage Collection ও Background Task-এর প্রভাব পড়তে পারে। তাই একই Input-এ বহুবার চালিয়ে Median, p95, Worst Case বিবেচনা করবে। Development Build বনাম Production Build আলাদা করে পরীক্ষা করবে।

```javascript
function benchmark(fn, iterations = 30) {
  for (let i = 0; i < 5; i++) fn(); // warm-up
  const samples = [];
  for (let i = 0; i < iterations; i++) {
    const start = performance.now();
    fn();
    samples.push(performance.now() - start);
  }
  samples.sort((a, b) => a - b);
  return { median: samples[Math.floor(samples.length / 2)], max: samples.at(-1) };
}
```

এটি শেখার জন্য Simple Benchmark; Professional Microbenchmark-এর বিকল্প নয়।

২৩.৪ Browser Rendering Pipeline

Browser সাধারণত JavaScript, Style Calculation, Layout, Paint ও Compositing-এর কাজ করে। বারবার DOM Read/Write মিশিয়ে করলে Forced Synchronous Layout হতে পারে।

```javascript
// Avoid repeated read-write loops
const widths = elements.map(el => el.getBoundingClientRect().width);
elements.forEach((el, i) => { el.style.width = `${widths[i] + 10}px`; });
```

`DocumentFragment`, DOM Batch Updates, CSS `transform` এবং `requestAnimationFrame()` সঠিক জায়গায় ব্যবহার করলে Rendering Work কমতে পারে। তবে `requestAnimationFrame` CPU-heavy কাজকে স্বয়ংক্রিয়ভাবে দ্রুত করে না।

২৩.৫ Debounce, Throttle ও Input Responsiveness

Search Input-এর প্রতিটি Keystroke-এ বড় Filter চালানোর বদলে Debounce করা যায়।

```javascript
function debounce(fn, wait = 200) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), wait);
  };
}
const onSearch = debounce(query => console.log('Search:', query), 200);
```

Debounce: শেষ Input-এর কিছু সময় পরে Run। Throttle: নির্দিষ্ট সময়ের মধ্যে Run Frequency সীমিত। Scroll Rendering-এর জন্য `requestAnimationFrame`-এ Schedule করা উপযোগী হতে পারে।

২৩.৬ Virtual Scrolling কেন?

১২,০০০টি Result থাকলে একসঙ্গে ১২,০০০টি Row DOM-এ রাখার বদলে দৃশ্যমান অংশ এবং কিছু Extra Row Render করা যায়। Fixed Height ৫৬ px, Viewport ৪৮০ px হলে সাধারণত কয়েক ডজন Row যথেষ্ট।

```javascript
const rowHeight = 56;
const overscan = 6;
const start = Math.max(0, Math.floor(scrollTop / rowHeight) - overscan);
const end = Math.min(total, Math.ceil((scrollTop + viewportHeight) / rowHeight) + overscan);
```

এখানে Spacer Full Scroll Height বজায় রাখে, আর Visible Rows `translateY` দিয়ে সঠিক অবস্থানে বসে। Variable Height, Focus Retention ও Screen Reader Accessibility-এর জন্য আরও উন্নত Design প্রয়োজন।

২৩.৭ Memory Management ও Garbage Collection

JavaScript Engine Reachable Object-কে ধরে রাখে। Unreachable Object Garbage Collection-এর যোগ্য হয়। Memory Leak সাধারণত এমন Reference থেকে আসে যা আর দরকার নেই, কিন্তু এখনো Reachable।

সাধারণ কারণ: Global Array-এ অনবরত Data জমা, Detached DOM Node ধরে রাখা, Cleanup ছাড়া Event Listener, `setInterval()` বন্ধ না করা, সীমাহীন Cache এবং Closure-এ বড় Data আটকে রাখা।

```javascript
function startTicker(element) {
  const id = setInterval(() => { element.textContent = new Date().toLocaleTimeString(); },
1000);
  return () => clearInterval(id); // cleanup when view is destroyed
}
const stop = startTicker(document.querySelector('#clock'));
// Component unmount / page teardown: stop();
```

`WeakMap` নির্দিষ্ট Object-keyed Metadata-র জন্য কাজে লাগে, কিন্তু সব Memory Leak সমাধান করে না। Memory Panel-এর Heap Snapshot Compare করে Retained Object খুঁজতে হবে।

২৩.৮ Web Workers, Lazy Loading ও Network

Main Thread-এ ভারী Calculation UI Block করতে পারে। Web Worker আলাদা Thread-এ CPU-heavy Work করতে পারে, তবে Worker DOM Access করতে পারে না এবং Message Transfer-এর খরচ আছে।

```javascript
const worker = new Worker('./worker.js', { type: 'module' });
worker.postMessage({ type: 'ANALYZE', records });
worker.addEventListener('message', event => console.log(event.data));
```

Network Optimization-এর জন্য Code Splitting (`import()`), Asset Compression, Browser Caching, Image Size ও Lazy Loading বিবেচনা করবে। অকারণে Lazy Loading করলে Critical Content দেরিতে দেখা দিতে পারে।

২৩.৯ Practical Project — TravelSearch Explorer

Project Folder: `source-code/chapter-23/`

```
chapter-23/
├── index.html
├── style.css
├── core.mjs
├── app.mjs
├── core.test.mjs
└── RUN.md
```

Features

- ১২,০০০টি Deterministic Fictional Travel Records
- Destination Search ও Filter
- Price/Rating/Title Sorting
- ১৮০ ms Debounced Search
- Fixed-height Virtualized List
- `performance.now()` দিয়ে Local Filter/Sort Timing
- Unit Tests for Filtering, Sorting, Window Range ও Memoization

Run

TERMINAL

```
cd source-code/chapter-23
node --test core.test.mjs
python -m http.server 8000
```

তারপর Browser-এ `http://localhost:8000` খুলবে। `file://` দিয়ে ES Modules চালালে Browser Security Restriction হতে পারে। `core.mjs`-এ Business Logic, `app.mjs`-এ Browser UI আলাদা করা হয়েছে।

গুরুত্বপূর্ণ Design Trade-off

এই Demo Fixed-height Virtualization শেখায়। এটি Production-ready Accessible Data Grid নয়। Search-এর পরে Scroll Reset করা হয়; Real Application-এ Keyboard Navigation, Focus Management, Empty State, Variable Height, Loading States এবং Screen Reader Behavior আলাদাভাবে Test করতে হবে।

২৩.১০ Performance Profiling Lab

১. Browser DevTools → Performance খুলে Search Input-এ দ্রুত টাইপ করে Record করো।
২. Main Thread Flame Chart-এ Long Task আছে কি না দেখো।
৩. Search Debounce বন্ধ করে আগের Measurement-এর সঙ্গে তুলনা করো।
৪. Virtualization বাদ দিয়ে ১২,০০০ Row Render করলে DOM Node Count ও Responsiveness কীভাবে বদলায়,

পরীক্ষা করো।

৫. Memory → Heap Snapshot নিয়ে Search-এর আগে-পরে Retained Object তুলনা করো।

৬. Device CPU Throttling ব্যবহার করে Low-end Device-এর কাছাকাছি পরিস্থিতি তৈরি করো।

Benchmark Report Template: Device, Browser Version, Dataset Size, Scenario, Median, p95, Memory, Observations, Change, Regression Risk। এই বই কোনো Universal Speed-up Percentage দাবি করছে না।

২৩.১১ Common Mistakes

ভুল	কেন সমস্যা	করণীয়
শুধু একবার Benchmark	Noise ও JIT প্রভাব	Multiple Samples
Sort-এ Original Array Mutate	Unexpected State Change	`[...items].sort()`
Unlimited Memoization Cache	Memory Growth	Bounded Cache / Eviction
`innerHTML`-এ User Input	XSS Risk	`textContent`
১০,০০০ DOM Row একসঙ্গে	Layout/Paint Cost	Virtualization / Pagination
Cleanup ছাড়া Listener	Memory Retention	Remove Listener / AbortSignal
শুধু Desktop-এ Test	Mobile Performance অজানা	Real Device / Throttling

২৩.১২ Exercises — ১০টি

১. ৫,০০০ Item-এর `find()` ও `Map.get()`-এর Performance Benchmark করো।

২. `normalizeQuery()`-তে Whitespace Trim ও Case-insensitive Search Test করো।

৩. `filterTrips()`-এ Price Limit যুক্ত করো।

৪. `sortTrips()` যেন Input Mutate না করে, Test লেখো।

৫. `windowRange()`-এ Scroll Position ০ হলে Correct Range নির্ণয় করো।

৬. List-এর শেষের দিকে Overscan যেন `total` অতিক্রম না করে, Test লেখো।

৭. Search Input-এ Debounce ব্যবহার করো।

৮. Event Listener Cleanup-এর একটি উদাহরণ লেখো।

৯. DevTools Performance Profile থেকে Long Task শনাক্ত করে নোট লেখো।

১০. Virtualized বনাম Full Rendering-এর তুলনামূলক Report তৈরি করো।

২৩.১৩ Sample Solutions

```javascript
// 2
normalizeQuery('  DHAKA ') === 'dhaka';
// 3
filterTrips(makeTrips(100), { maxPrice: 4000 });
// 4
const input = makeTrips(10);
const before = input.map(x => x.id);
sortTrips(input, 'price-desc');
console.assert(JSON.stringify(before) === JSON.stringify(input.map(x => x.id)));
// 5
windowRange({ scrollTop: 0, rowHeight: 50, viewportHeight: 200, total: 100, overscan: 2 });
// 6
windowRange({ scrollTop: 4950, rowHeight: 50, viewportHeight: 200, total: 100, overscan: 2
}).end === 100;
// 8
const controller = new AbortController();
window.addEventListener('resize', () => console.log('resized'), { signal: controller.signal });
controller.abort(); // remove registered listener
```

Exercise 1, 7, 9 ও 10-এর ফল Device/Browser অনুযায়ী ভিন্ন হবে; এগুলো Hands-on Lab। `core.test.mjs`-এ Exercises 2–6-এর Runnable Verification আছে।

২৩.১৪ Knowledge Check — ১০টি MCQ

১. Optimization-এর প্রথম ধাপ কী? A) Measure B) Rewrite everything C) Add cache D) Remove tests

২. `performance.now()` কী মাপে? A) High-resolution elapsed time B) Network speed only C) Heap only D) CPU cores

৩. `sort()` ডিফল্টভাবে কী করে? A) Mutates array B) Always immutable C) Deletes values D) Caches results

৪. Debounce-এর কাজ কী? A) Wait after latest input B) Always run every ms C) Increase memory D) Block UI

৫. Virtualization কেন? A) Fewer mounted rows B) More DOM nodes C) More CSS D) Remove sorting

৬. Long Task-এর সাধারণ Threshold কত? A) 50 ms B) 1 ms C) 5 sec D) 10 sec

৭. Memory Leak-এর উদাহরণ কোনটি? A) Forgotten interval B) Pure function C) Local const D) Static text

৮. Web Worker কি সরাসরি DOM Modify করতে পারে? A) Yes B) No C) Only CSS D) Only images

৯. p95 কী নির্দেশ করে? A) 95th percentile B) Average only C) Memory limit D) Request count

১০. Lab Benchmark কেন যথেষ্ট নয়? A) Real devices differ B) JS is always same speed C) Cache never matters D) Browser never changes

Answers ও Explanations

1. **A** — Measurement ছাড়া Bottleneck বোঝা যায় না।
2. **A** — High-resolution Time Difference পাওয়া যায়।
3. **A** — `sort()` Original Array বদলায়; Copy করে Sort করা নিরাপদ।
4. **A** — শেষ Event-এর পরে Delay দিয়ে Work চালায়।

5. **A** — শুধু দৃশ্যমান Row Render করা হয়।
6. **A** — Main Thread-এর ৫০ ms-এর বেশি Task Long Task হিসেবে চিহ্নিত হয়।
7. **A** — Cleanup ছাড়া Timer Reference ধরে রাখতে পারে।
8. **B** — Worker DOM Access করতে পারে না।
9. **A** — Samples-এর ৯৫ শতাংশ ওই Value বা তার নিচে।
10. **A** — Hardware, Browser ও Network ভেদে ফল বদলায়।

২৩.১৫ Chapter Recap ও Next Chapter

Performance Optimization হলো **Measured Engineering**। Algorithm Complexity, DOM Rendering, Event Scheduling, Memory Lifetime ও Network—সবকিছু একসঙ্গে বিবেচনা করতে হয়। TravelSearch Explorer-এর মাধ্যমে ১২,০০০ Record-এ Filter/Sort, Debounced Input ও Virtual Scrolling প্রয়োগ করা হয়েছে। পরবর্তী অধ্যায় **TypeScript Integration, Static Typing এবং Professional Development Workflow**।